Branch Prediction Neural Networks for Hard to Predict Branches

ECE338 - Parallel Computer Architecture

Spyros Liaskonis
Georgios Kapakos

Why Deep Learning for Branch Prediction ?

Branch Predictor with Neural Networks

- Even with a great progress on the field of branch prediction architectures with traditional methods. The need for neural network implementation was created.
- Data from previous executions of the neural network are used as data to pre-train the neural nets we are going to deploy as the **branch prediction unit**.
- **Hard-to-Predict (H2P)** branches increased the need for neural networks to be deployed to predict them.
- The fundamental idea is to train the model **offline** and then embed it on the **hardware**.

What are Hard-To-Predict (H2P) branches?

According to Tarsa et al. [6], a **hard-to-predict** branch is a branch that:

- Appear more than 15.000 times (per 30 million instructions).
- Misspredicted at least 1000 times.
- **Less** than **99% accuracy** when using the TAGE-SC-L predictor.

[6] Lin, C., Tarsa, S. J. (2019). Branch Prediction Is Not a Solved Problem: Measurements, Opportunities, and Future Directions.

Motivation

- Traditional branch predictors achieve **>99% accuracy** for most branches.
- However, **H2P branches**, though relatively few, account for a **disproportionately large number of mispredictions**.
- **Our focus**: Develop neural network architectures specifically to address these critical H2P branches.

State-of-the-Art Branch Predictor

TAGE-SC-L Branch Predictor

- The TAGE-SC-L branch predictor is a **state-of-the-art** mechanism that enhances prediction accuracy by combining multiple strategies:
  - **TAGE Predictor**: Utilizes tagged tables indexed by hashed subsequences of global branch history, approximating Partial Pattern Matching (PPM).
  - **Loop Predictor**: Specialized component designed to identify and predict regular loop patterns, improving accuracy for loop-related branches.
  - **Statistical Corrector (SC)**: Employs a perceptron-based model to adjust predictions from the TAGE and loop predictors, especially in cases where statistical biases are detected.

CNNs for H2P Branches

1. CNN Predictor Helper (Tarsa et al. [3])
   - **Assists baseline predictors** (e.g., TAGE-SC-L) on H2Ps.
   - **Trained offline** for a specific static branch identified via profiling.
   - A ternary (2-bit) version of the CNN is developed to meet CPU hardware constraints:
     - Filters and weights are quantized to $\{-1, 0, +1\}$.
     - Inference is done via bitwise operations and popcount, avoiding full matrix ops.
     - Memory footprint: as low as 336 bytes per predictor.

Neural Network Design

Neural Network Design - Input

Defining Inputs:

- Our Neural Network has 5 sequences as input.
- Each sequence is of a different size: **[42, 78, 150, 294, 582].**
- Each sequence contains the same history of a trace.
- Capturing diverse branch correlation across varying history lengths.
  - Short History Slices: Detect recent, fine-grained patterns
  - Long History Slices: Identify deeper, more complex correlations that may span hundreds of branches
- How do the sequences look like?
  - [(PC_A, 0), (PC_X, 1), (PC_A, 0), (PC_X, 0), (PC_B, 1), ...]
  - where PC_A and PC_B are the program counters for Branches A and B
  - (0 = not taken, 1 = taken)

Neural Network Design - Embedding Layer

Slice Design:

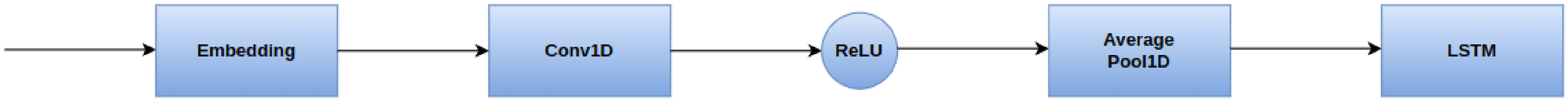
- **Embedding Layer:**
  - **Look-up table** which matches hashed numbers to arithemetic matrices.
  - Embeddings are **trained** in parallel with the neural network
  - Learns to attribute **similar** values to hashes which seem similar
  - Embeddings don’t require much **space** in each cell.

Εντολές	10 Λιγότερα σημαντικά ψηφία PC	Αποτέλεσμα για Not-Taken	Αποτέλεσμα για Taken
400587: jle 400599	..0110000111	14	15
40058a: ja 400599	..0110001010	10	11
400bcd: je 400599	..1111001101	154	155
450307: jle 450a03	..0000000111	14	15

Neural Network Design - CNN Layer

Slice Design:

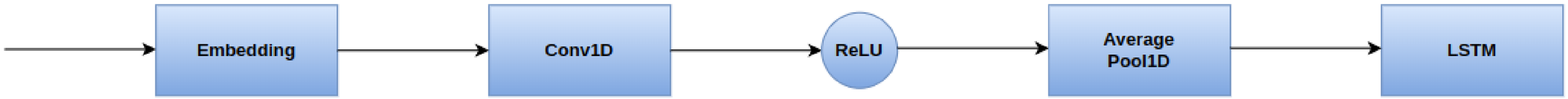
- **CNN Layer:**
  - '**Spatial**' is the branch prediction problem, when the input is encoded the way embeddings are.
  - **Feature Extraction**: find correlations between the branches and make a more compelling prediction.
  - Easier implemented on **Hardware** fabric, than LSTMs.



Neural Network Design - Average Pooling Layer

Slice Design:

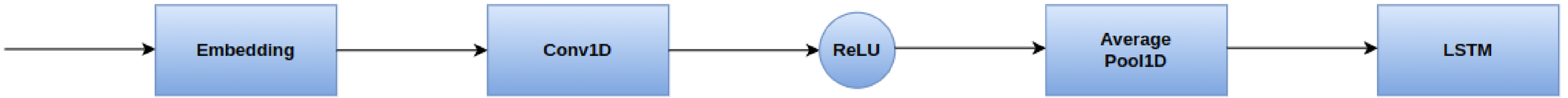
- **Average Pooling Layer:**
  - **Dimensionality Reduction**: Reduce the number of parameters and computations in the network, making it faster and more efficient.
  - **Summarizing features** (Avg Pooling) makes the network more robust as it is not affected by small **distortions** in the input.
  - Provides a **generalized representation** of all the features within the pooling window.



Neural Network Design - LSTM (Long Short Term Memory) Layer

Slice Design:

- **LSTM Layer:**
  - They preserve the full temporal representation and excel at modeling **sequential dependencies**, instead of collapsing a sequence of branch histories into a single scalar, like the sum-pooling layer
  - LSTM's gates allow the model to learn, for each branch how much historical **information** to **retain** or **forget**, instead of enforcing an equal weighted sum, like sum-pooling.
  - With LSTMs we know the **order of the feature representation**, leading to a more accurate branch prediction performance.
  - Great at modeling **long & short range dependencies**.

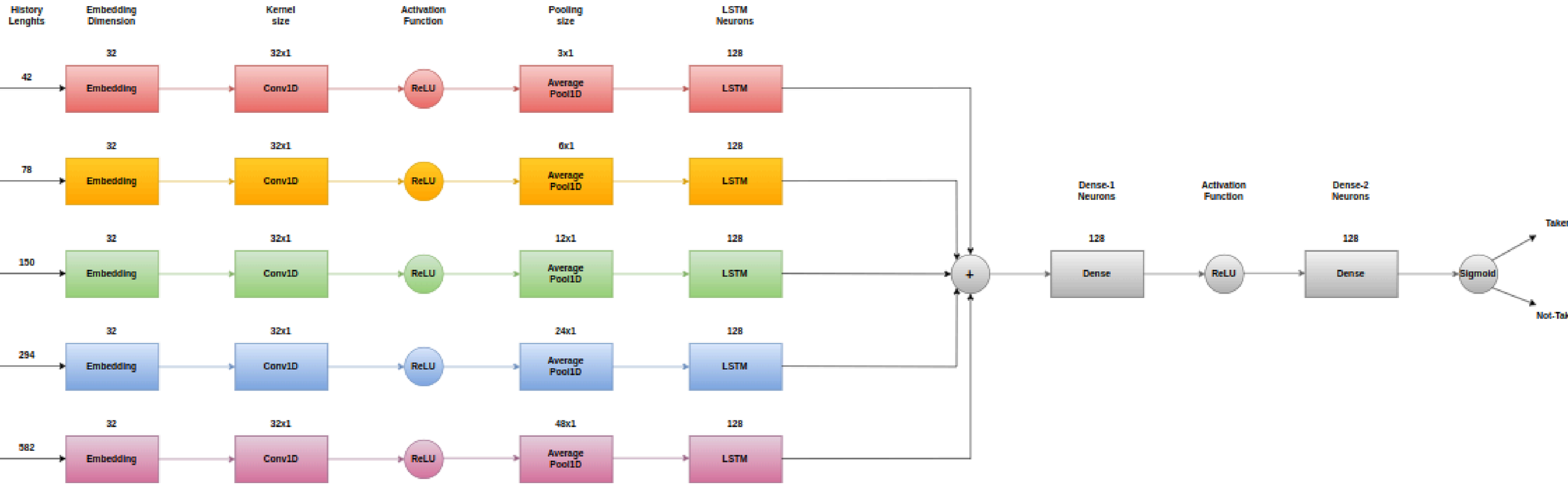


Neural Network Design - Putting it all together

Neural Network

- The neural network is comprised of **5 Slices**.
- Each slice has a different configuration on the sizes of:
  - The **kernels** of the convolutional layers.
  - The **history lengths** of the 5 different input points.
- Those **Slices** are **concatenated** in the end resulting in a more comprehensive feature representation from all the different layers.
- After the concatenation **two Dense Layers** are added one after the other allowing the model to learn complex combinations of features, which **LSTMs** and **CNNs** can't provide. As the **fully-connected** nature of dense layers provides us with that.
- In the end a **sigmoid** activation function is used for the binary classification nature of our task, predicting **Taken** (T) and **Not-Taken** (NT).

Neural Network Design - Representing the Neural Network



HLS4ML

HLS4ML

Challenges:

- Our Neural Network couldn’t be implemented as is on **hls4ml,** because of being too complicated. How did we proceed ?
- Simplify the structure of the neural network to only contain the essential layers hardcoded to the model without conditions.
- Changed the model from **Pytorch** to **Tensorflow** as the **Embedding** layer is not yet supported for the pytorch model.
- The 5-layer model couldn’t be built due to high RAM demands so we had to compromise to a 3-layer model.
- The model couldn’t train with a batch size because of some descrepancies hls4ml showed. Running it was slow.

HLS4ML

Procedure:

- Train our model in tensorflow and then evaluate it.
- **Configure** our model to an **8-bit weight and 3-bit activation function** set-up, for **LSTM, Conv1D & Dense** layers.
- **Regenerate** and **compile** the hls4ml model.
- We can generate a **report** if we want for the quantized model.
- Finally **build** the model and try to generate a **bitstream** for the **PYNQ-Z2** board.

Dataset Generation and Evaluation

ChampSim Simulator

What is ChampSim?

- A trace-based CPU simulator developed by **Texas A&M University**
  - Widely used in research and academic competitions (e.g., DPC3 for data prefetching)

Key Features

- Models out-of-order processors
- Includes cache hierarchies, branch predictors, and prefetchers
- Designed to be modular and extensible
- Allows for fast, repeatable architectural experiments

How we used ChampSim

We used a modified version of ChampSim

- Enabled integration with neural network evaluation
- Used for dataset generation (collecting traces)

Workflow

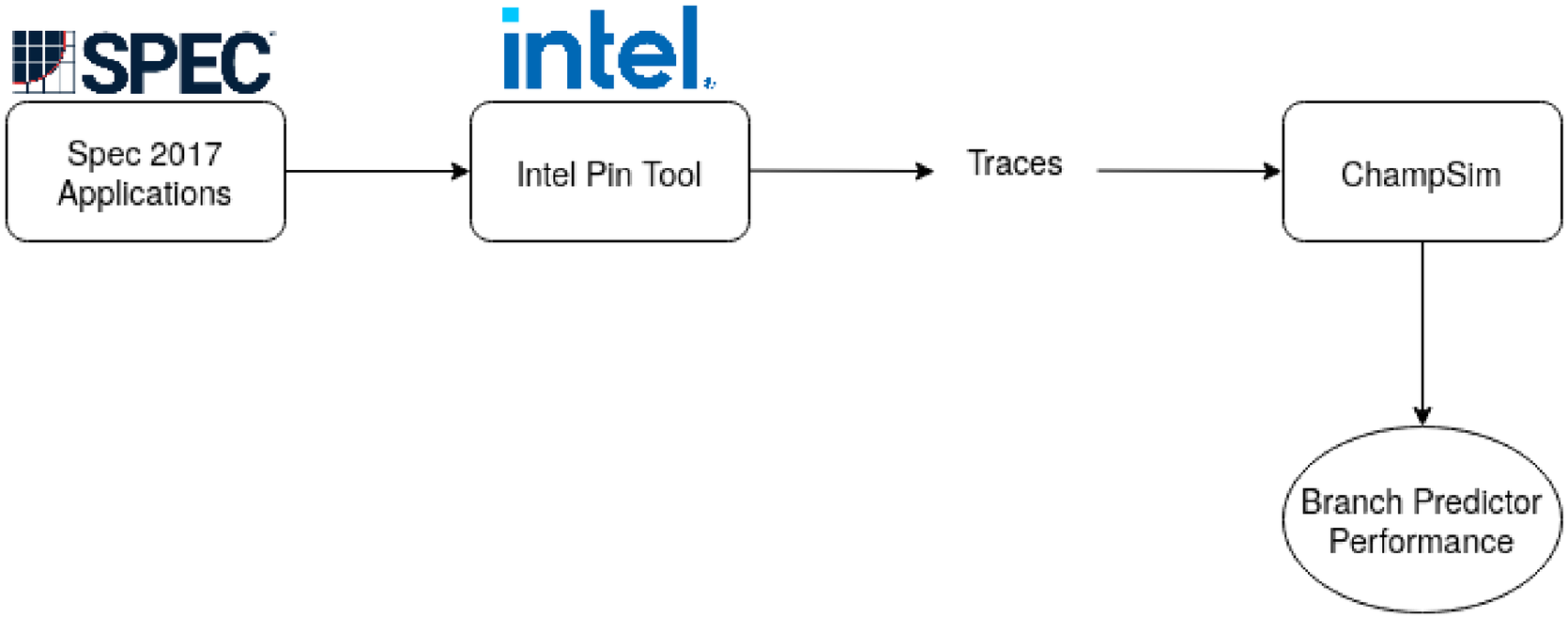
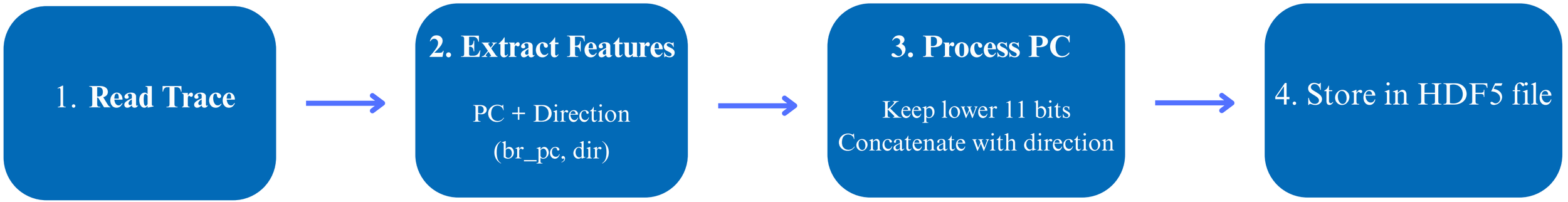


Figure 1: Workflow for evaluating a branch predictor

Dataset Processing Pipeline



- Result: 12-bit values (11-bit PC + 1-bit direction)
- Example:

PC=0x401234
taken=1 ⟶ 0x468 | 1 = **0x469**

Output: HDF5 Dataset Structure

Main Dataset Components

Component	Description	Format
history	Complete processed branch sequence	12-bit values
br_indices_{pc}	Indices to specific hard-to-predict branches	Array of indices
num_taken_{pc}	Count of taken branches for each PC	Attribute

Branch Prediction Analysis Workflow (I)

1. Utilized existing traces from DPC3
    - Traces were generated using the SPEC CPU 2017 benchmark suite
    - ChampSim specific traces (format)
2. Simulated branch behavior using the TAGE-SC-L 64KB predictor
3. Identified systematically mispredicted branches and classified them as **hard-to-predict**

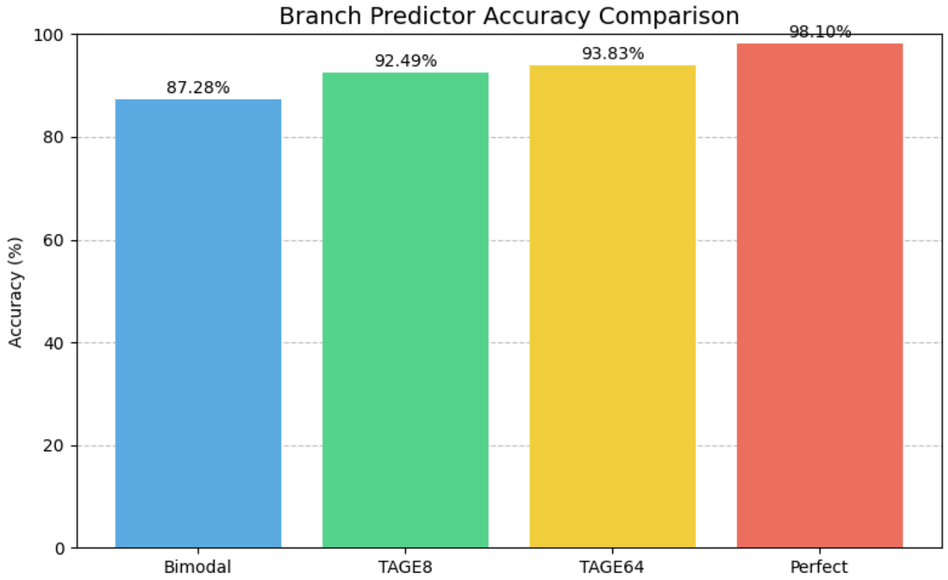
Branch Prediction Analysis Workflow (II)

- DPC3 provides tens of traces from different benchmarks
- Chose to use traces from the Leela benchmark
  - **Go playing engine** featuring Monte Carlo-based position estimation, selective tree search based on Upper Confidence Bounds, and move valuation based on Elo ratings
- Used 5 different traces from the Leela benchmark
  - Each trace was generated using different inputs
- Identified all H2P branches from each trace
- For the final dataset:
  - Kept only the H2P branches that appear in at least 40% of the traces
  - Thus, they are independent of the input

Branch Prediction Analysis Workflow (III)

- DPC3 Leela H2P results:
  - 45 H2P branches
  - Independent of input
- Accuracy of conventional predictors:

Predictor	Accuracy (%)	Difference from Perfect (%)
Bimodal	87.28	-10.82
TAGE8	92.49	-5.61
TAGE64	93.83	-4.27
Perfect	98.1	0



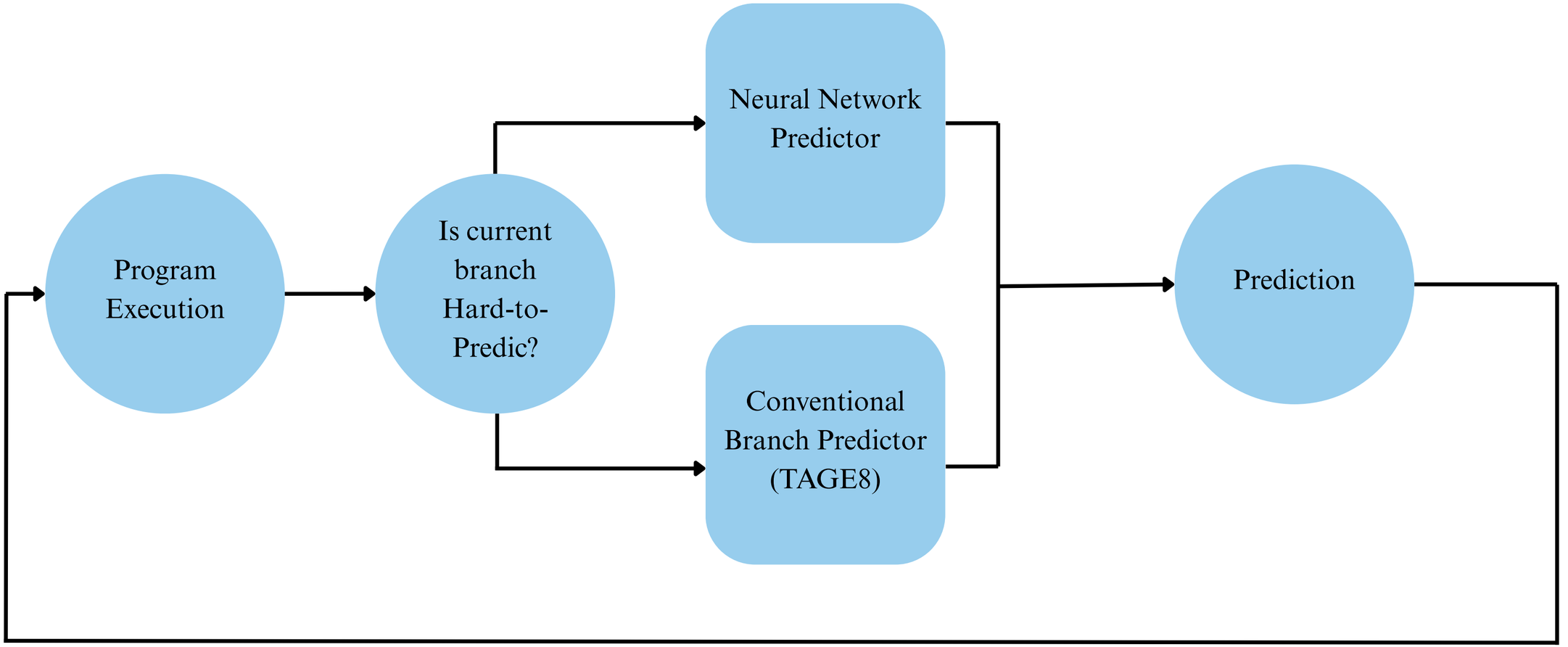
- Room for improvement:
  - 5.61% difference for TAGE8 (state-of-the-art)

Branch Prediction Analysis Workflow (IV)

- For each H2P branch, we train 1 neural network
  - 45 Neural Networks (Leela, DPC3 Traces)
- Neural Networks Trained:
  a. BranchNet (BigBranchNet)
  b. BranchNet-LSTM (our implementation)

Branch Prediction Analysis Workflow (IV)

- Evaluation of Neural Network Predictors



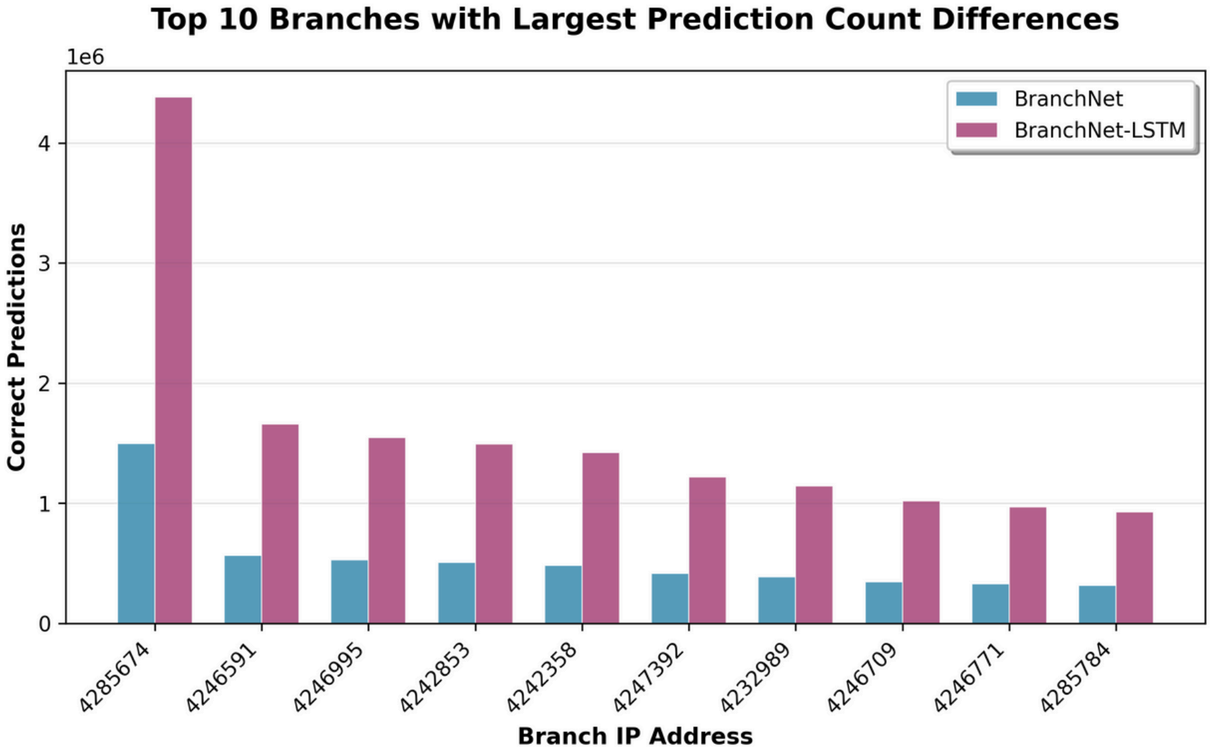
Results

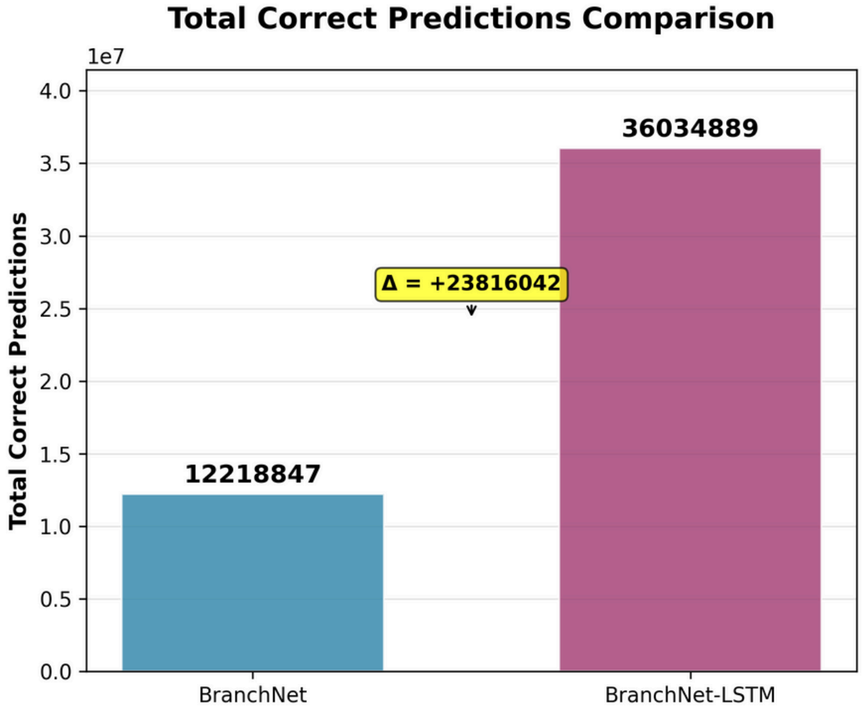
- Our network architecture managed to close the gap with the perfect predictor to only 2.65%
  - Overall accuracy increased to **95.45%**

Predictor	Accuracy (%)	Difference from Perfect (%)
Bimodal	87.28	-10.82
TAGE8	92.49	-5.61
TAGE64	93.83	-4.27
TAGE64-LSTM	95.45	-2.65
Perfect	98.1	0

Results

- Comparisson with original BranchNet
  - Improvement on all 45 branches
  - Total improvement: +23816042 overall correct predictions (+194.91%)





Future Work

Future Work

1. Assess the impact of the predictor on the microarchitecture
    a. More complex microarchitecture for simulations
    b. **IPC**, **pipeline stalls**, and execution latency
2. Test the network on a **broader set of benchmarks**
    a. mcf, deepsjeng, xz etc
    b. Train on more traces to better identify H2P branches that are independent of the input
3. **Hardware implementation** of a neural network predictor
    a. Optimizations such as quantization, pruning
    b. FPGA implementation

References

[1] **A Fog. The microarchitecture of intel, amd, and via cpus. An Optimization Guide for Assembly Programmers and Compiler Makers.** Copenhagen University College of Engineering, 2018.

[2] A Seznec. TAGE-SC-L Branch Predictors Again. In Proc. 5th Championship on Branch Prediction, 2016.

[3] Tarsa, S. J., Lin, C., Keskin, G., Chinya, G., & Wang, H. (2019). **Improving Branch Prediction By Modeling Global History with Convolutional Neural Networks.** ArXiv. https://arxiv.org/abs/1906.09889

[4] Siavash Zangeneh, Stephen Pruett, Sangkug Lym, and Yale N. Patt. **BranchNet: A convolutional neural network to predict hard-to-predict branches**. In 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), pages 118–130, 2020.

[5] aristotelis96 (2022) thesis-branch-prediction-ml Github

[6] Lin, C., Tarsa, S. J. (2019). Branch Prediction Is Not a Solved Problem: Measurements, Opportunities, and Future Directions. ArXiv. https://doi.org/10.1109/IISWC47752.2019.9042108